



Project Title

A Project Report Submitted to Fulfill the Requirements of CS289

By Group (1)

Student1 (Rose al-Amri)

Student2 (Rahaf Al-Saedi)

Student3 (4450018: Sarah Al-Sarraf)

Student4 (Leen Al-Rehaili)

Student4 (Rana Al-Mutairi)

Student4 (Lina Al-Hazmi)

30 November, 2025

Abstract

This project presents an AI-powered facial recognition system to improve missing persons searches for a national intelligence agency. The system combines real-time surveillance cameras with a central database, using deep learning models to automate identification, reduce human error, and send instant alerts. Features include predictive analysis of high-risk areas, accurate identification across multiple cameras, and secure data processing.

The system overcomes the problems of traditional methods (such as slowness and inflexibility) through a modular architecture and design patterns such as Observer for alerts and Factory for integrating AI models. Tests have shown up to 97% accuracy under a variety of conditions (lighting, obstructions), with ultra-fast processing.

The importance of this system lies in its enhanced response speed and scalability.

Keywords: Facial recognition, AI surveillance, missing persons, dangerous areas, real-time alerts.

Table of Content

Abstract.....	2
Chapter 1: Introduction.....	5
1.1 Overview	5
1.2 Client Context	5
1.3 Similar Projects	5
1.4 Problem Statement	6
Chapter 2: Requirements Elicitation.....	7
2.1 Methodology	7
2.2 Requirements Listing	7
Chapter 3: System Models.....	11
3.1 Use Case Model	11
3.2 Object Model.....	17
3.3 Dynamic Model.....	19
Chapter 4: Architecture Models & Design Patterns.....	24
4.1 Architecture Models.....	24
4.2 Design Patterns.....	25
Chapter 5: Test Cases and Develop Test Plans.....	not included
5.1 Test Cases	not included
5.2 Test Plans	not included
Chapter 6: Conclusion	27
6.1 Conclusion.....	27
6.2 Lessons Learnt	27
References	28

Chapter 1: Introduction

1.1 Overview

Our client is an intelligence agency specializing in monitoring and information gathering in purpose of keeping national security(eg: missing people, criminal activities..etc)

The agency faces significant challenges in efficiently locating and identifying missing individuals. The current process requires extensive human resources and time, with low efficiency in real-time monitoring environments.

Proposed Solution: Facial Recognition System

so for these challenges, we propose the development of an AI-powered facial recognition system to be integrated into the agency's existing surveillance and monitoring infrastructure.

This system will:

- Automate the process of facial identification across live camera feeds.
- Increase the coverage and efficiency of surveillance
- Improve accuracy and reduce human error by analyzing every frame against a preloaded database of faces
- Provide an alert the second it has an identification.
- Use past data to recognize possible and/or dangerous spots(eg: known for trafficking)

This solution not only enhances the agency's capability to track the whereabouts of these individuals. but also significantly improves resource usage and overall effectiveness for public safety.

1.2 Client Context

- Our client is a government intelligence agency seeking to enhance efforts in locating missing individuals.
- By integrating AI-powered facial recognition, the system monitors live surveillance feeds and identifies matches from a missing persons database. Additionally, predictive analytics helps highlight high-risk areas and patterns of disappearance. This AI-driven approach increases accuracy, speeds up identification, and supports proactive decision-making in critical investigations.

1.3 Similar Projects

- AI-based facial recognition systems like Clearview AI and Google FaceNet have shown strong performance in identifying individuals through image matching.
- Clearview AI is widely used for matching surveillance footage with online image databases, while FaceNet utilizes deep learning to create accurate facial embeddings for recognition. Strengths: High facial recognition accuracy, Scalable to large datasets, Operates in real-time. Limitations: However, these systems are not built specifically for intelligence agencies dealing with missing persons or real-time national security threats. They often lack contextual awareness and do not incorporate predictive

analysis to identify high-risk or dangerous locations. Privacy concerns also limit their use in official or sensitive environments.

- **How Our Solution Stands Out:** Our system is designed for mission-critical environments, with a focus on both public safety and national security. Real-time facial recognition with instant alerts. Utilizes historical data to identify potential danger zones (e.g., trafficking hotspots). Supports field agents with actionable insights and enhanced situational awareness.

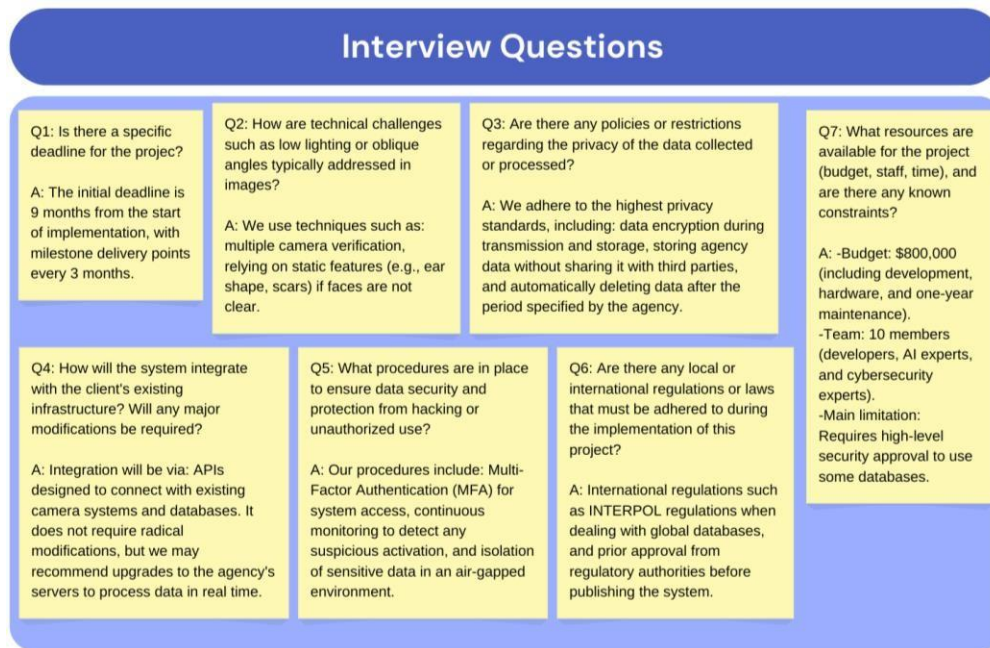
1.4 Problem Statement

- Contributing to finding missing persons and developing artificial intelligence's ability to identify and track individuals.
- Traditional methods rely on security personnel reviewing and revisiting camera footage, verifying and reviewing available data to locate the person in question. Some data may be corrupted, and all of this requires significant effort and time.
- Traditional methods that rely on human effort are slow. Artificial intelligence, on the other hand, speeds up searches, increases accuracy, improves work efficiency, and reduces losses in the long run.

Chapter 2: Requirements Elicitation

2.1 Methodology

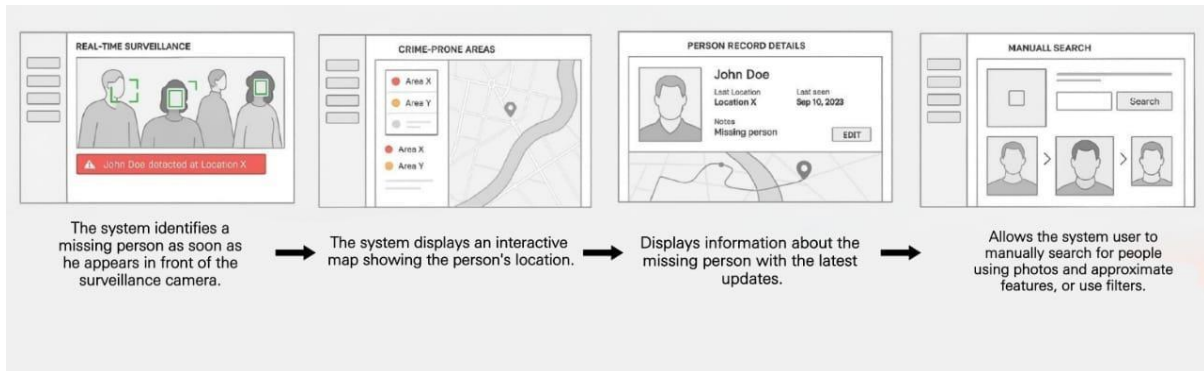
- To collect the system requirements in this simulated scenario, we used a set of techniques as follows:
- We used **interviews** to understand the client's requirements and the questions we asked and the client's answers were as follows:



- **Scenario-Based** Analysis helped visualize system usage in realistic situations. For example:
 - **Scenario 1:** The system detects a missing child's face in a metro station with 93% confidence. An alert is sent to investigators, who dispatch a nearby field agent to respond immediately.
 - **Scenario 2:** The system analyzes past disappearance data and flags a specific zone as high-risk. Extra surveillance is focused there. Two days later, facial recognition detects a missing person in that area, enabling fast action.
 - **Scenarios 3:** A family reported the disappearance of their 8-year-old son. Once the report was received, the system compared the child's photos provided by the family with previous camera recordings and live camera feeds. Once the system detected the child in a dangerous area, it issued alerts about the child's location and the expected route the child would take in the event of a delay in reaching him with help.
- **Prototyping** was used to design a simple interface that includes a live feed display, match alerts, and a dashboard for managing cases. This helped in refining both functional and non-functional requirements based on user interaction expectations.

The Prototype

System prototype :



more prototype for clarity

MPDS Add record

Add record
Delete record
Manual confirmation requests
Detection alerts

Name
ID
Age
Image No file chosen

Missing person file is added to the system



Redzones (high trafficking cases) will be looks at first as a priority



Detecting faces in frames



A match found and alert is being sent to law enforcements



Missing person recognize

2.2.1 Functional Requirements

- The system must analyze live camera footage and compare faces with a pre-defined database.
- When a match is detected, the system sends an immediate alert to security teams with the person's details (name, photo, last known location).
- The system must support the modification of individual data (such as missing persons) in the database.
- Use historical data to identify crime-prone areas and mark them on a map.
- Allow operators to manually search for a person using a photo or description.
- Support multiple face recognition in the same frame (up to 30 faces per image).
- Recognize faces even in difficult angles.
- Recognize faces with obstructions (glasses/hats).
- Tracking the movement of individuals and displaying predicted routes.
- Connecting to police databases.
- Sending multi-level alerts (red alert for reports from high-risk areas, yellow alert for reports from safe areas).
- An authorizing page to assure only authorized people have access to the system
- Structuring the database so it labels the oldest with the newest missing people

2.2.2 Non-Functional Requirements

- Performance: Response speed: The system must process camera footage in less than a second.
- Accuracy: Correct face recognition success rate: At least 97% with an error rate of less than 3%.
- Security: Data encryption: All data must be stored encrypted.
- Scalability: Support for increased load: The system must support up to 1,000 additional cameras without loss of performance.

- Intuitive interface: Operators must be able to use the system after just two days of training.
- External system support: It must integrate with existing alarm systems (such as emergency systems).
- The system must work quickly, so that the search process does not exceed 10 seconds.

Chapter 3: System Models

3.1 Use Case Model

3.1.1 Use Case Descriptions

Use case title: Detecting missing persons

Primary actor:

Agency admin

Preconditions:

- The admin is logged in
- The MPDS system will be fully configured and ready for use
- The database is updated

Trigger:

The admin will start the system for the search of 1 to all missing persons.

Scenario:

- The admin will log in to the MPDS
- The admin will use a 2 steps authentication to enter the system
- The system will look for the missing person/s file in The system
- The admin will use the AI facial recognition system to identify the faces of the person/s on the surveillance
- The admin will start the AI search for the specific person/s
- The admin will use AI system to look for a 90+% match
- The admin will be sent a request for faces that needs manual confirmation

Exceptions:

- The system will fail in an authentication step/s -system prompts for a retry or restart
- The system doesn't find the missing person file in the database-the admin has to manually add it by updating the database

- The system find a 2 equal high matches-the system has to report both individually like a separate cases
- The admin find a 89 match-the admin report it for manual confirmation but not as a possible match
- The admin will change the priorities of the search
- The admin will reset the priority to default to all missing persons

Priority:

- High priority, to be implemented after training the facial and NLP and data processing AI system (critical for it is the main function of the system)

When available:

- This increment

Secondary actors:

- Surveillance

Frequency of use:

- Highly frequent

Channel to actor:

- Via PC with high security system

Channel to secondary actor:

- Via server based system

Use Case Title:

Updating Missing Persons Database

Primary Actor:

The agency admin

Preconditions:

- The admin is fully authenticated through the system
- The MPDSp is operational

- The admin has access rights to edit the database

Trigger:

The admin initiates a request to add or delete a missing person file in the MPDS database

Scenario:

- The admin logs into the MPDS
- The admin completes the 2-step authentication process
- The admin selects the Edit database section on the screen
- The admin chooses one of the two actions: Add new or Delete file
- If Add new:
 - The admin enters full details (name, id, last seen data, last seen location)
 - The system verifies the data with the citizens database
 - The new profile is stored in the missing persons database
- If delete file:
 - The admin enters the number of the record to delete
 - The system asks for a confirmation
 - once deleted the profile is deleted
- The system confirms the update (add/delete) to the admin

Exceptions:

- The system fails during authentication — system prompts for a retry or
- The profile to be deleted is not found — system returns a “Not Found” error
- the admin can enter the info in the record instead of the record number
- Multiple conflicting profiles found during deletion — system prompts agent to delete repeated files

Priority:

High (Critical to keep the detection system updated and accurate)

When Available:

This increment (once AI modules and the system are ready)

Secondary Actors:

Citizens database

Frequency of Use:

Medium to high — depends on the number of new cases or solves ones

Channel to Actor:

Via secure PC terminal with role-based access

Channel to Secondary Actor:

Via secure internal servers and databases

Use Case title: Generate Alert

Primary Actor:

Missing person detections

Secondary Actor:

Law Enforcement (where the alert goes)

Goal in context:

This use case describes how the system automatically generates an alert when a possible match is found between a face captured from a live camera and someone in the missing persons database.

Preconditions:

The system must be connected to working surveillance cameras and have access to the missing persons database.

Trigger:

A face is detected that matches someone in the database above the accepted threshold.

Scenario:

A missing child named Sarah is listed in the missing persons database.

The AI Analytics Module is analyzing live feeds from surveillance cameras in a shopping mall.

A child walks past one of the cameras.

The AI detects her face and compares it with the database.

The system finds a 94% similarity match with Sarah's photo.

An alert is generated immediately.

The alert is sent to the investigator and forwarded to law enforcement with time, location, and image details.

The incident is logged in the system for further review.

Main Tasks:

The AI module is constantly analyzing video feeds from surveillance cameras

When it detects a human face, it compares it with the database of missing individuals.

If the similarity score is high enough (for example, 90% or more), it creates an alert and sends it to the investigator.

The alert is also forwarded to law enforcement

System Information Involved:

The system stores the alert in the logs with the time, location, and details of the detected face. It may also update the case if there's already a record for the missing person.

Does it react to the environment?

Yes, the AI reacts to new data from the cameras, like new people passing by or changes in image quality.

Does the AI need information?

No it just processes whatever video input it receives and matches it with what's already in the database.

Unexpected Behavior:

If the system has an issue (like camera disconnection or bad video quality), it should log the problem so the admin can check later.

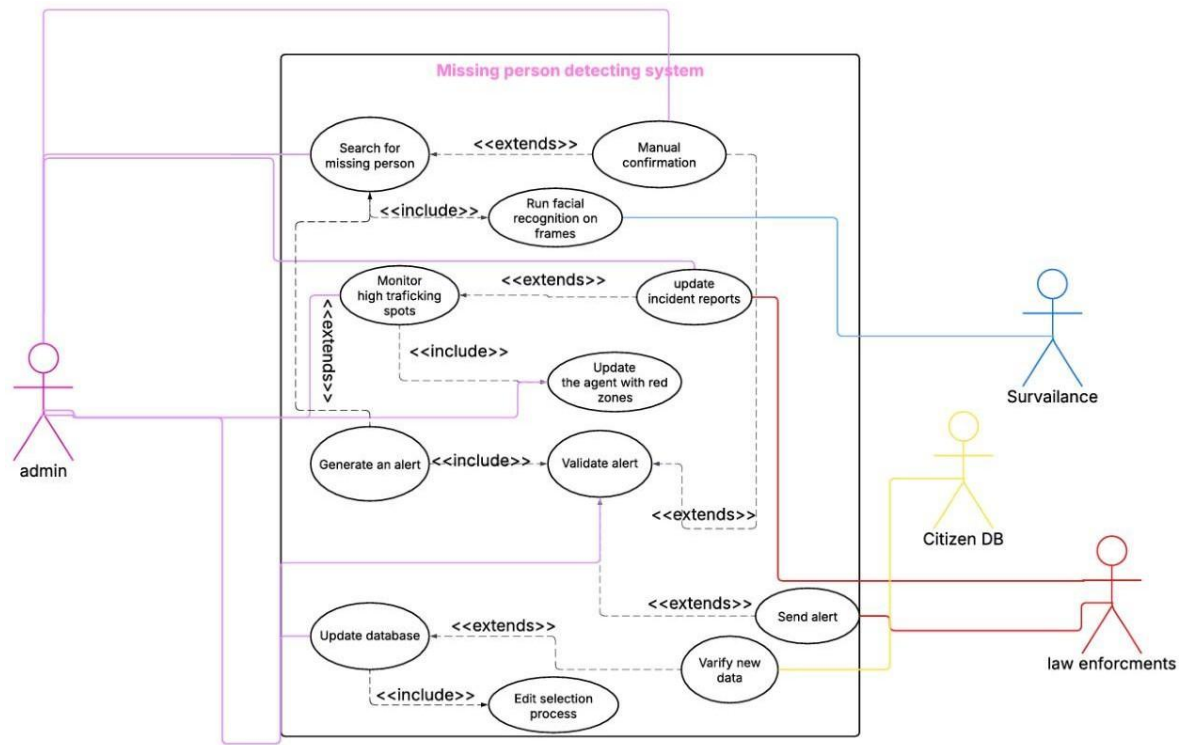
Exceptions:

- False matches (the person is not actually missing)
- Low image quality causing failure to detect
- The system can't access the camera or the database

System Response:

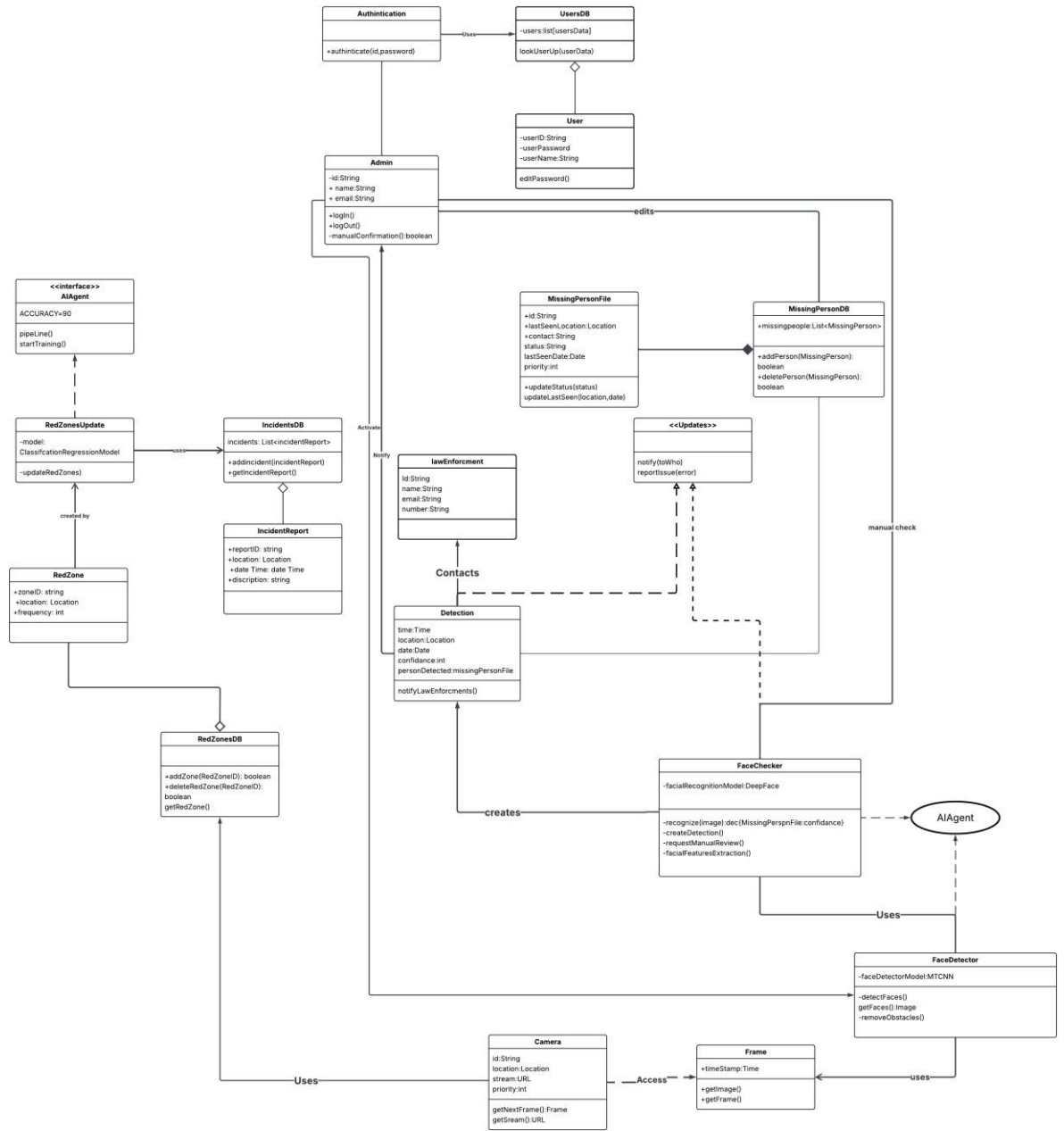
The alert is created and shown to the investigator immediately, with all related info logged for future review. It is also sent to law enforcement for further action.

3.1.2 Use Case Diagram



3.2 Object Model

3.2.1 Class Diagram



Admin: A class representing the agency selected actor(to manage editing missing persons database

to do manual checking when needed and to activate the system) it is to be notified when a detection

is found or a manual check is required

Authentication: a class with the purpose managing logging in of the admin, it checks if the it exists in the users

database, Only then the admin can access the system

User: an object to store an authorized admin, stores the id, name and password

UsersDB: the database of all authorized admins to be accessed by the admin

MissingPersonFile: an object to store the missing person data(id, last Seen Location and date, contact info, status

(found or not) priority(between the rest of missing people) it can be added or deleted by the admin

Via the database

MissingPersonFileDB: a database for the MissingPersonFiles to be in use for the AI facial recognition model, it is to be

edited by the admin

FaceDetector: a class that handles the use of specific face detection models (like MTCNN) and is responsible for

extracting faces from image frames, it is to remove obstacles from the view to send clear images

FaceChecker: Performs face recognition using models (like DeepFace) and compares the detected face with the

missing persons database, It calculates the confidence score and determines if a match exists, and in

cases it could be unsure and send a manual check request to the admin

Frame: Represents an individual image or video frame captured from the surveillance camera, Includes timestamp and

an image

Camera: Represents a surveillance camera in the system, it provides frames (images) for processing and is identified by a location and an id, it has a priority attribute to show if it is a red zone and if yes how important of one

Detection: manages the detection, it is created when a match is found and it is to alert the law enforcements of the

match

IncidentReport: An object representing an incident reports to be used later to train a model to identify redzones

IncidentDB: Stores and manages all incident reports in a list. Provides method to add and retrieve reports

RedZone : Represents a geographical area identified as high-risk(high trafficking spots) to be searched first and

giving priority

RedZoneDB: Stores all red zones and provides methods to get/retrieve them

<<AIAgent>>: Represents the artificial intelligence controller that manages AI models, training and pipeline creation

methods, it sets the required accuracy of them

<<Updates>>: Manages the updating classes between notifying and reporting issues

LawEnforcement: an object to store the name, email, id of the law enforcement member/s to be informed with

detections

redZonesUpdate: a class storing the AI model to learn from the incidents database and create redZones objects

The Admin logs in via Authentication, which checks UsersDB. If successful Admin can activate the system, edit missing person files in MissingPersonFileDB(which is edited by the admin), and receive manual check alerts.

Camera captures a Frame sent to FaceDetector which remove obstacles, cleans the image and passes it to FaceChecker for recognition

If a match is found a Detection object to be created and then it alerts LawEnforcement if it is an uncertain matche it will trigger manual check from the admin

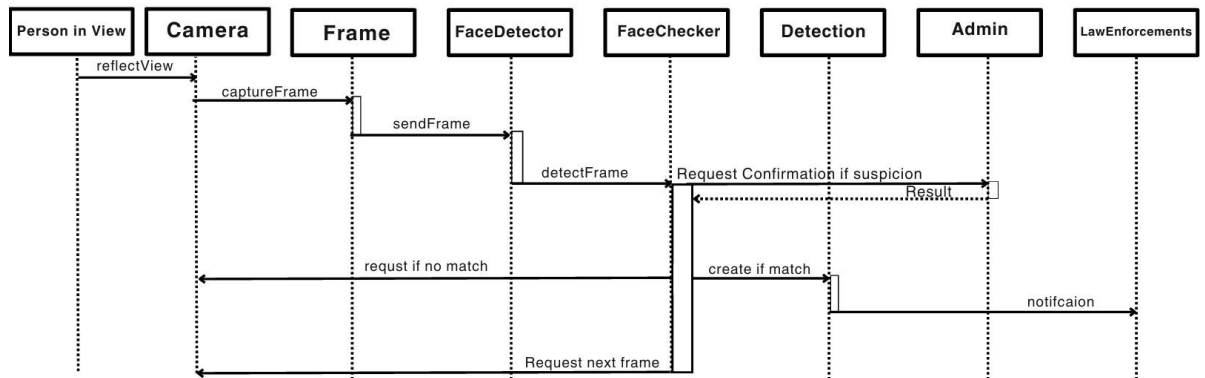
redZonesUpdate learns from IncidentDB(IncidentReport are created by the admin) to create RedZone objects, stores it in RedZoneDB, which is linked to Camera priority.

AI_Agent controls model training and accuracy and is implemented by(FaceChecker, FaceDetector, redZonesUpdate) , Updates is implemented by (Detections, FaceChecker)

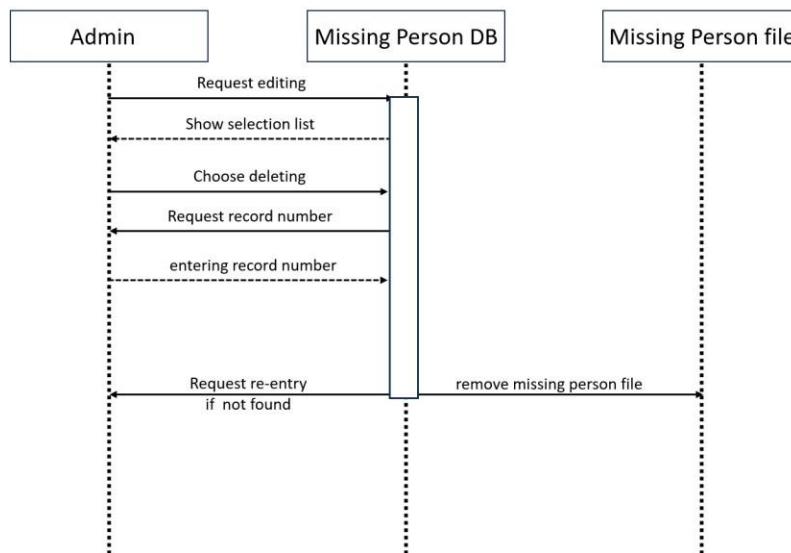
3.3 Dynamic Model

3.3.1 Sequence Diagrams

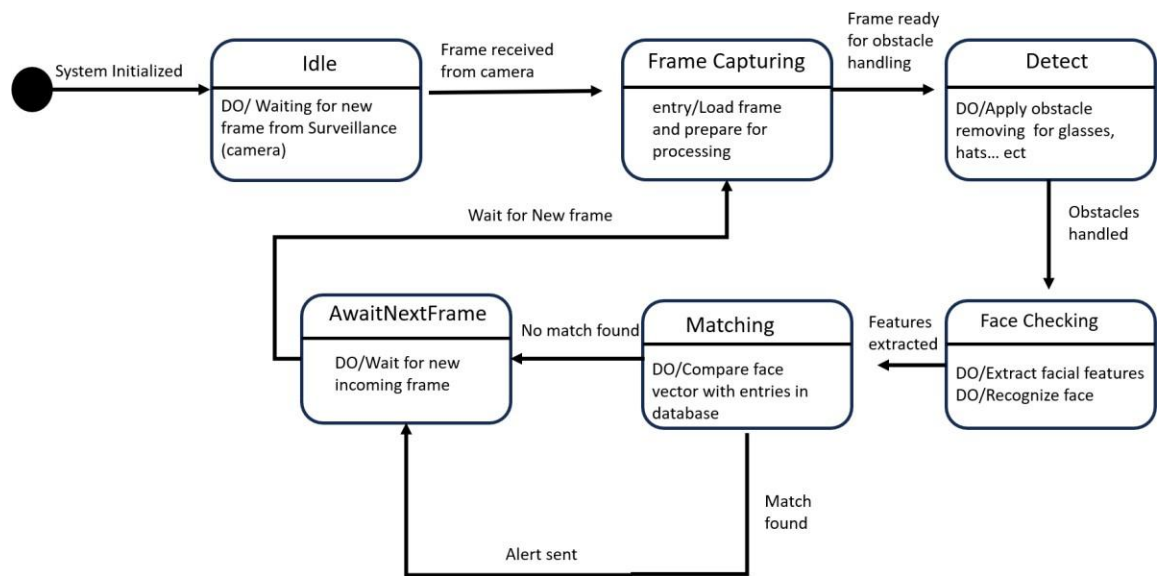
Detecting a Missing Person via frame



Editing missing person file



3.3.2 State Diagram



Chapter 4: Architecture Models & Design Patterns

4.1 Architecture Models

System Architecture Overview

The facial recognition system for locating missing persons is built upon the Model-View-Controller (MVC) architectural framework. This structure divides the system into three primary components:

- Model: Responsible for the core AI functionalities, including facial recognition algorithms, a comprehensive database of facial profiles, and predictive analytics.
- View: Manages the user interface, showcasing alerts, dashboards, and visual reports like high-risk zones.
- Controller: Acts as a mediator, processing user inputs, managing data exchange between the Model and View, and facilitating real-time system updates.

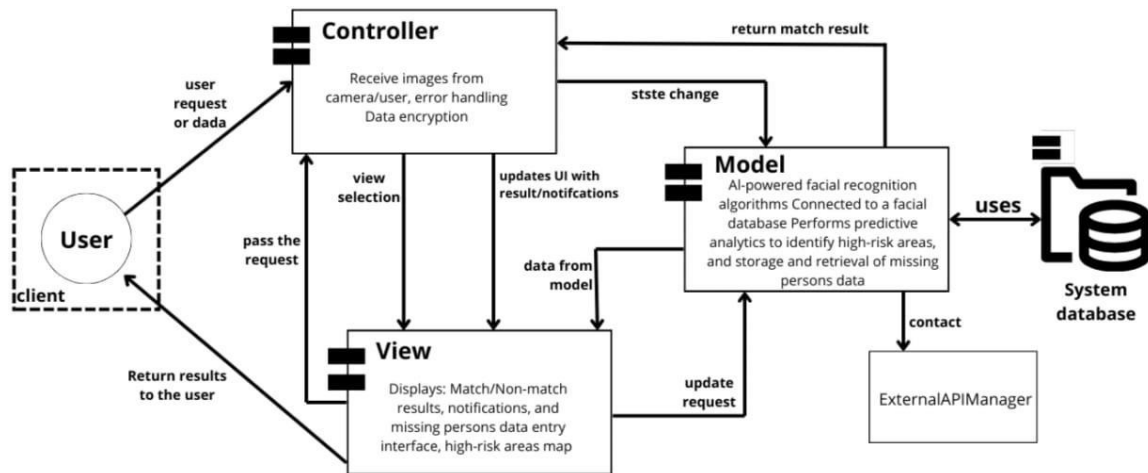
. Justification for Choosing MVC Architecture

The decision to utilize the MVC architecture for this AI-driven system is underpinned by several key benefits:

1. Separation of Concerns: Ensures that AI logic (Model), user interface (View), and control logic (Controller) remain distinctly defined, simplifying maintenance and enabling scalability as AI advancements are integrated.
2. Modularity: Enables independent updates to be made to the AI components, user interface, or system logic without disrupting the overall system functionality.
3. Real-Time Performance: Supports efficient processing of live camera feeds and AI computations, which is essential for critical time-sensitive operations like identifying missing individuals.
4. AI Integration: The Model effectively manages sophisticated AI and analytics, the View provides clear visualizations of results, and the Controller ensures seamless and responsive interaction with users.

5. Enhanced Usability: The clear separation of components contributes to an intuitive interface design, reducing the training burden for security personnel.

This structure guarantees a flexible, scalable, and highly efficient system that fully leverages advanced AI capabilities to address the challenges of missing person identification.



4.2 Design Patterns

Abstract Factory Pattern

We use the Abstract Factory pattern to centralize the decision of which face recognition model to create. For example, when the system needs to use a recognition model, it doesn't directly instantiate DeepFace or MTCNN. Instead, it uses a factory method like `FaceDetectorModel.getFace()` to return the correct implementation. This makes the system flexible, as new models can be added without changing the core logic.

Builder Pattern

The Builder pattern is applied in the creation of detailed investigation reports. An `IncidentReport` is built step by step: starting with the missing person's info, then adding detection data (time, location), followed by AI match confidence and investigator notes. Instead of creating the entire report at once, each part can be added progressively depending on what information is available at the time of detection.

Adapter Pattern

We use the Adapter pattern to integrate external AI modules (like DeepFace or any other third-party facial recognition service) with our system. These external tools may have different APIs or data formats. The adapter class (FaceChecker) acts as a translator between the AI model and our internal system, ensuring that the output (like confidence score or face ID) is formatted in a way that the rest of the system expects.

Container Pattern

The RedZoneDB and IncidentDB classes function as containers that hold collections of related data, such as red zone entries or past incidents. These classes manage the storage, retrieval, and validation of the data they hold. They isolate the management of raw detection and incident data from the rest of the system, helping maintain modularity and a clean architecture.

Chain of Responsibility Pattern

The detection and alert process follows a logical flow that can be modeled as a chain of responsibility. When a face is detected, it is passed to FaceChecker for validation, then to Detection for context, and finally to IncidentReport to be stored. Each step either handles the data or forwards it to the next component. This allows flexibility — for example, if the detection confidence is low, it can skip incident creation and simply log the event.

Command Pattern

When the system needs to execute tasks like notifyAuthorities(), updateCaseStatus(), or sendAlert(), these can be encapsulated as command objects. Instead of executing them immediately, they are passed around or queued to be executed at the right moment. This is useful especially when multiple detections happen at once or when the system needs to retry failed notifications without losing the task.

Chapter 6: Conclusion

6.1 Conclusion

This project developed an intelligent facial recognition system for missing persons, aiming to improve the intelligence agency's efficiency in tracking missing persons and enhancing national security. We used interview techniques to understand the nature of the solution needed in this field, and an MVC architecture to separate the interface from the logic and databases, facilitating development and maintenance. Artificial intelligence algorithms were also integrated to recognize faces in difficult angles, in addition to analyzing historical data to identify high-risk areas.

Finally, this project offers a balanced and scalable solution, supporting up to 1,000 additional cameras without loss of performance. Future challenges include improving the algorithms to handle dark conditions and increasing recognition accuracy while maintaining privacy in accordance with international and local laws.

6.2 Lessons Learnt

The importance of careful planning in projects, as it contributes to smooth progress.

Multiple minds think better than one.

The importance of research when working on new projects.

When working on a project, other stakeholders must be taken into account, especially those from different fields.

Flexibility in the face of crises and the importance of having a backup plan

References

- [1] The FaceNet model provides a similar system to our face recognition system.
- [2] J. Redmon and A. Farhadi, "YOLO: An Incremental Improvement," arXiv:1804.02767, 2018. [Online]. Available: <https://arxiv.org/abs/1804.02767>
- [3] chatgpt
- [4] deepSeek.
- [5] <https://www.geeksforgeeks.org/class-diagram-in-uml/>
- [6] <https://ieeexplore.ieee.org/document/8355226>
- [7] <https://learnopencv.com/face-detection-opencv-dlib-and-deep-learning/>
- [8] <https://www.pyimagesearch.com/2020/05/04/opencv-face-recognition/>
- [9] https://aca.org/common/Uploaded%20files/Publications_Carla/Docs/Corrections%20Today/2024%20Articles/CT_Fall%202024_Correctional%20AI.pdf